

AULA · ARQUITETURA FRONT-END

Next.js, BFF e TanStack Query

Quando criar BFF, quando bater direto na API, onde encaixar o TanStack Query e quais são *de fato* os padrões que a indústria usa hoje.

Pré-requisito: nenhum. Vamos do zero.

Stack-alvo: Next.js 14/15 (App Router) + TanStack Query v5

Atualizado em: Maio de 2026 · fontes oficiais ao final

Sumário

- 1 Por que isso parece tão confuso (e você não é burro)
 - 2 Os 4 atores em jogo
 - 3 As 4 formas de buscar dados no App Router
 - 4 Quando você PRECISA de BFF (e quando não)
 - 5 Server Actions vs Route Handlers
 - 6 O padrão profissional: as 3 camadas e o atalho do Server Component
 - 7 Onde entra o TanStack Query (e por quê)
 - 8 Server Component + TanStack: o padrão "prefetch + hydrate"
 - 9 O wrapper na prática: um `apiFetch` tipado
 - 10 Receitas prontas (copia e cola mental)
 - 11 Anti-patterns que vão te queimar
 - 12 Cheatsheet final
 - 13 Fontes
-

1 Por que isso parece tão confuso

Spoiler: **você não é burro**. Você está aprendendo um framework que mudou fundamentalmente em 2023 e continua evoluindo. Antes do App Router, "BFF no Next" era basicamente *"escreva um arquivo em `pages/api`"*. Fim. Hoje você tem **4 ferramentas que se sobrepõem** (Server Components, Server Actions, Route Handlers, Client Components com fetch) e cada blog escolhe uma e diz que é "o jeito certo".

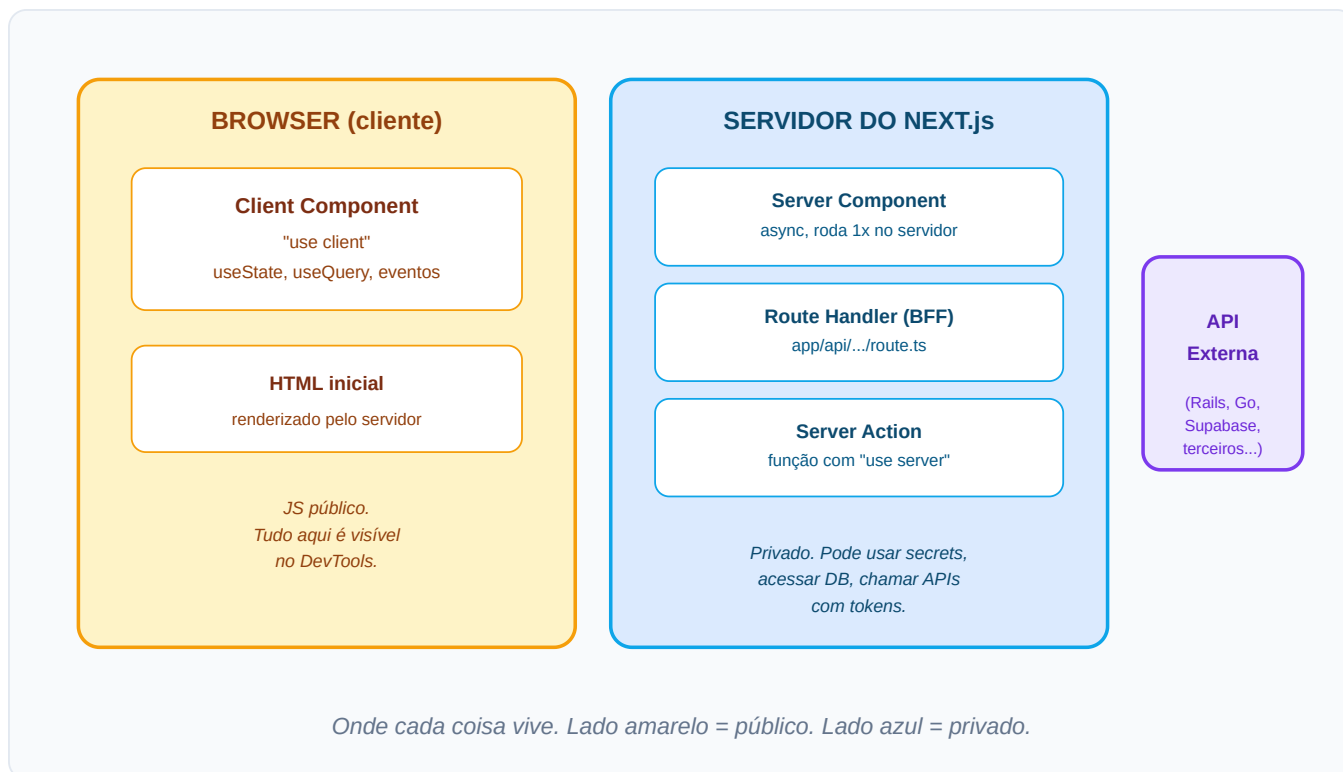
Eles não estão todos errados — estão respondendo perguntas *diferentes*. A confusão é porque ninguém te diz **qual pergunta cada ferramenta responde**. É isso que vamos resolver aqui.

TERMOS PRIMEIRO

BFF (Backend for Frontend): uma camada de backend *dedicada* ao seu front-end. Em Next.js, ela vive dentro do próprio projeto, geralmente em `app/api/.../route.ts` (Route Handlers). Não é um servidor separado — é parte do Next rodando no servidor.

2 Os 4 atores em jogo

Antes de qualquer decisão, tenha esse mapa mental claro:



Fixe isso: **tudo que está no lado amarelo o usuário consegue ler**. Se você botar uma `API_KEY` num client component, ela vai parar no DevTools de qualquer pessoa que abrir o site. Isso é o motivo número 1 para existir BFF.

3 As 4 formas de buscar dados

Forma	Onde roda	Use quando...
Server Component async function Page()	Servidor (1x por request)	Dado de leitura, mostrado no primeiro paint , sem interação. Ex.: página de blog, detalhe de produto.
Route Handler app/api/x/route.ts	Servidor (HTTP endpoint)	Você precisa de uma URL chamável: webhook, mobile, BFF para client com TanStack, OAuth callback.
Server Action "use server"	Servidor (POST mágico)	Mutação chamada do próprio front (criar post, deletar item, login com form).
Client fetch useQuery / useEffect	Browser	Dado que muda com interação: filtros, paginação, polling, dado por usuário logado, real-time.

A PERGUNTA-MESTRE

"O dado é parte do paint inicial ou aparece depois?"

Parte do paint → Server Component.

Aparece depois (interação, refetch, polling) → Client + TanStack Query, que vai bater num Route Handler (BFF) ou direto na API externa, dependendo das regras da próxima seção.

4 Quando você PRECISA de BFF

Esquece "boas práticas abstratas". Tem 5 motivos **concretos** pra criar uma rota em `app/api/.../route.ts`. Se nenhum se aplica, *não crie*.

4.1 — Você tem um segredo (API key, token de serviço)

Se a chamada precisa de uma chave que *não pode* vazsar pro browser (Stripe secret key, token de admin, chave do OpenAI), **obrigatório** BFF. O client chama `/api/seu-recurso`, e o Route Handler chama a API externa injetando a chave a partir do `process.env`.

4.2 — A API externa não tem CORS configurado pro seu domínio

Browser bloqueia. Servidor não. Se o backend é um sistema legado, uma API de terceiros que recusa seu Origin, ou qualquer coisa que dê `CORS error` no console, o BFF resolve porque a chamada sai de servidor pra servidor.

4.3 — Você quer agregar/transformar antes de mandar pro front

Sua tela precisa de *dados de 3 microsserviços*? Sua API legada devolve um JSON bizarro que o front tem que mastigar? Faça isso uma vez no BFF — não em cada componente. É *literalmente* o motivo pelo qual o padrão se chama "Backend **For Frontend**".

4.4 — Você precisa de auth com cookie httpOnly

Cookie `httpOnly` não é lido por JavaScript (essa é a graça — protege contra XSS). Quem lê o cookie e troca por um token pra API externa é o BFF.

4.5 — Webhooks, OAuth callbacks, integrações server-to-server

Stripe, GitHub, Twilio mandando um POST pra você? Isso é `route.ts`, ponto. O browser não está envolvido.

✓ Crie BFF se...

- Tem secret no meio do caminho
- CORS bloqueando
- Agrega de várias APIs
- Cookie httpOnly de auth
- Webhook / callback

✗ Não crie se...

- É só um proxy "burro" sem secret
- A API externa já tem CORS ok
- O endpoint é público mesmo
- Server Component já resolve
- Só vai duplicar a URL

MITO QUE PRECISA MORRER

"Sempre passe tudo pelo BFF, é mais seguro" → **falso**. Se sua API externa é pública e tem CORS configurado, fazer o front passar por `/api/proxy` só adiciona um hop, latência extra, e mais código pra manter. Segurança não vem de "passar pelo BFF", vem de *esconder o que precisa ser escondido*.

5 Server Actions vs Route Handlers

Ambos rodam no servidor. Ambos podem mutar dados. Quando usar qual? A regra é direta:

A REGRA DOS 30 SEGUNDOS

- **Só o seu próprio front chama?** → *Server Action*. É mais simples, tem type-safety automático, integra com forms.
- **Alguém fora chama? (mobile, webhook, outro serviço, ou client via TanStack Query)** → *Route Handler*. Você precisa de uma URL HTTP de verdade.

Server Action (exemplo)

```
// app/posts/actions.ts
"use server";

export async function createPost(formData: FormData) {
  const title = formData.get("title");
  await db.posts.create({ data: { title } });
  revalidatePath("/posts");
}

// app/posts/new-post-form.tsx (client)
<form action={createPost}>
  <input name="title" />
  <button>Criar</button>
</form>
```

Route Handler (exemplo)

```
// app/api/posts/route.ts
import { NextResponse } from "next/server";

export async function GET() {
  const posts = await db.posts.findMany();
  return NextResponse.json(posts);
}

export async function POST(req: Request) {
  const body = await req.json();
  // validar com zod, autenticar, etc
  const post = await db.posts.create({ data: body });
  return NextResponse.json(post, { status: 201 });
}
```

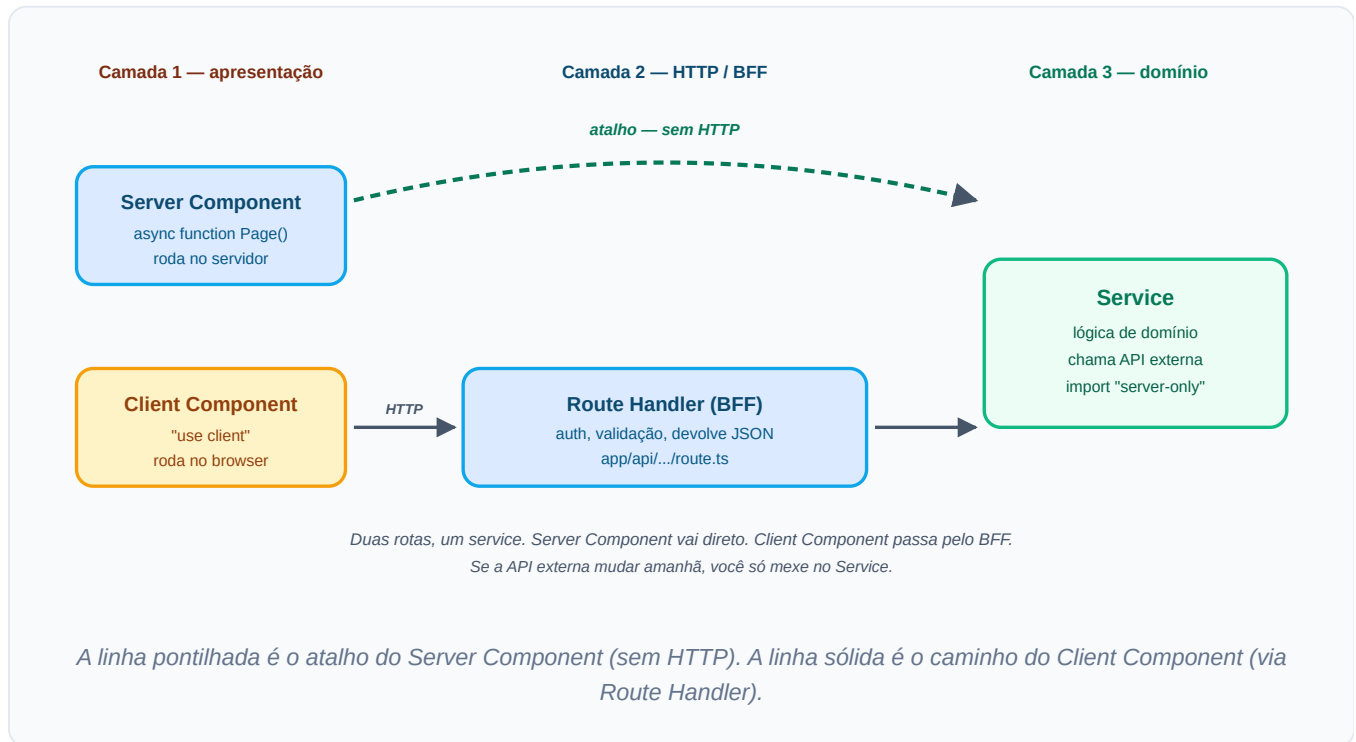
CUIDADO

Server Action **parece** uma função normal mas, na real, é um endpoint POST público. O `"use server"` não adiciona autenticação. Sempre valide e autorize lá dentro, igual você faria num Route Handler.

6 O padrão profissional

O padrão que times maduros usam e que a documentação do Next empurra é **3 camadas** — **Service, Route Handler, UI** — mas com uma sutileza importante: **Server Component e Client Component acessam o Service de jeitos diferentes**.

Server Component pula o Route Handler e importa o service direto (atalho seguro porque já está no servidor). **Client Component** não pode importar o service (vazaria secrets pro browser), então fala com o Route Handler, que aí sim chama o service.



Service: lógica de domínio

```
// lib/services/posts.ts — NUNCA importado por client component
export async function getPosts() {
  const res = await fetch(`${process.env.API_URL}/posts`, {
    headers: { Authorization: `Bearer ${process.env.API_TOKEN}` }
  });
  if (!res.ok) throw new Error("upstream error");
  return res.json() as Promise<Post[]>;
}
```

Route Handler: só HTTP + orquestração

```
// app/api/posts/route.ts
import { getPosts } from "@lib/services/posts";

export async function GET() {
  const posts = await getPosts();
  return Response.json(posts);
}
```

Server Component: usa o service direto, sem HTTP

```
// app/posts/page.tsx – Server Component
import { getPosts } from "@lib/services/posts";

export default async function Page() {
  const posts = await getPosts(); // in-process, zero HTTP
  return <PostList posts={posts} />;
}
```

POR QUE ISSO É BOM

Server Component **não precisa** chamar `fetch("/api/posts")`. Isso seria um HTTP "loopback" inútil — o Next chamando o próprio Next. Importe o *service* direto. O Route Handler existe pro *client*.

7 Onde entra o TanStack Query

TanStack Query (antigo React Query) **só faz sentido no lado do cliente**. Server Component já é síncrono do ponto de vista do React — você só `await`. Não tem nada pra "gerenciar".

No client, ele resolve coisas que `useEffect` + `useState` faz mal:

- **Cache compartilhado** entre componentes (3 componentes pedem o mesmo dado = 1 request).
- **Refetch automático** ao voltar pra aba, ao reconectar a internet.
- **Stale-while-revalidate**: mostra cache instantâneo, revalida no fundo.
- **Invalidation** declarativa após mutação (atualiza só o que mudou).
- **Loading / error / isFetching** sem você gerenciar a mão.

REGRA

Se um dado **aparece depois da hidratação e pode mudar** (filtros, paginação, polling, dado por usuário), use TanStack Query. Se aparece no primeiro paint e é estático até navegar de novo, use Server Component.

Setup mínimo

```
// app/providers.tsx
"use client";
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import { useState } from "react";

export function Providers({ children }) {
  const [client] = useState(() => new QueryClient({
    defaultOptions: { queries: { staleTime: 60_000 } }
  }));
  return <QueryClientProvider client={client}>{children}</QueryClientProvider>;
}

// app/layout.tsx (server)
export default function RootLayout({ children }) {
  return <html><body><Providers>{children}</Providers></body></html>;
}
```

POR QUE USESTATE E NÃO NEW QUERYCLIENT() SOLTO

Se você fizer `const client = new QueryClient()` no top-level do módulo, todos os usuários vão compartilhar o mesmo cache no servidor — vazamento de dados entre sessões. `useState` garante **um por sessão**.

Usando no componente

```
// components/posts-list.tsx
"use client";
import { useQuery } from "@tanstack/react-query";
import type { Post } from "@services/posts.service";

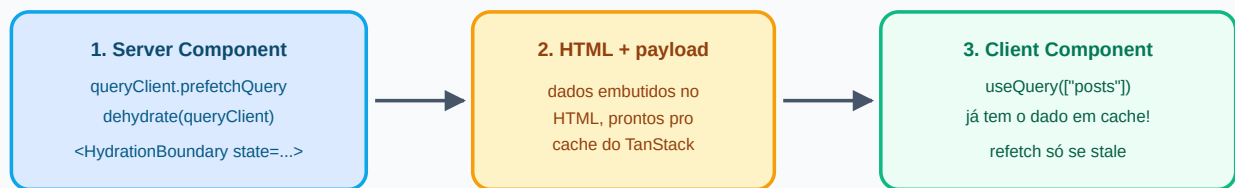
// helper tipado – o <T> força você a dizer o que espera de volta
async function apiFetch<T>(url: string): Promise<T> {
  const res = await fetch(url);
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return res.json() as Promise<T>;
}

export function PostsList() {
  // useQuery<TData, TError> – data fica Post[] | undefined, error fica Error
  const { data, isPending, error } = useQuery<Post[], Error>({
    queryKey: ["posts"],
    queryFn: () => apiFetch<Post[]>("/api/posts"),
  });
  if (isPending) return <p>Carregando...</p>;
  if (error) return <p>Deu ruim: {error.message}</p>;
  // data aqui é Post[] – autocomplete em p.id e p.title
  return <ul>{data.map(p => <li key={p.id}>{p.title}</li>)}</ul>;
}
```

Note que `queryFn` bate em `/api/posts` — ou seja, no Route Handler (BFF). **Pode** bater direto na API externa se ela for pública e tiver CORS ok. Mas se passar pelo BFF dá pra centralizar auth, retry, transformação e nunca expor secrets.

8 O padrão "prefetch + hydrate"

Esse é o "*goleação*": você quer SEO/first paint do Server Component e o cache esperto do TanStack no client. Dá pra ter os dois.



First paint instantâneo + interatividade total. Sem flash de "loading...".

Server prefetcha, manda o cache pronto pra hidratar no client.

Página (Server Component) — prefetcha e hidrata

```
// app/posts/page.tsx
import { QueryClient, HydrationBoundary, dehydrate } from "@tanstack/react-query";
import { getPosts } from "@lib/services/posts";
import { PostsList } from "@components/posts-list";

// queryKey exportada e tipada como tupla literal –
// garante que server e client usem EXATAMENTE a mesma key
export const POSTS_QUERY_KEY = ["posts"] as const;

export default async function Page() {
  const queryClient = new QueryClient();

  // getPosts é tipado como () => Promise<Post[]>,
  // o tipo flui pro cache automaticamente
  await queryClient.prefetchQuery({
    queryKey: POSTS_QUERY_KEY,
    queryFn: getPosts,
  });

  return (
    <HydrationBoundary state={dehydrate(queryClient)}>
      <PostsList />
    </HydrationBoundary>
  );
}
```

Componente cliente — usa a mesma queryKey

```
// components/posts-list.tsx
"use client";
import { useQuery } from "@tanstack/react-query";
import type { Post } from "@lib/services/posts";
import { POSTS_QUERY_KEY } from "@app/posts/page";

export function PostsList() {
  const { data } = useQuery<Post[]>({
    queryKey: POSTS_QUERY_KEY, // MESMA key do prefetch
    queryFn: () => fetch("/api/posts").then(async r => {
      if (!r.ok) throw new Error(`HTTP ${r.status}`);
      return r.json() as Promise<Post[]>;
    });
  });
}
```

```

    }},
  });
  // no primeiro render, data já vem populado e tipado da hidratação
  return <ul>{data?.map(p => <li key={p.id}>{p.title}</li>)}</ul>;
}

```

COMO A TIPAGEM FLUI

Note que o tipo `Post` é definido **uma vez** no service (`lib/services/posts.ts`) e é reusado em todo o resto. `getPosts` já é tipado como `() => Promise<Post[]>`, então o `prefetchQuery` herda esse tipo. No client, você passa o genérico explícito (`useQuery<Post[]>`) porque o `fetch` nativo retorna `any` — o cast acontece dentro do `queryFn`. Resultado: `data` vem como `Post[] | undefined`, com autocomplete em `p.id` e `p.title`.

QUANDO VALE A PENA ESSE PADRÃO

Quando o dado precisa **aparecer rápido** (SEO, perceived performance) e precisa ser **refetchável/invalidável** depois (filtros, mutações, polling). Se é só leitura estática → Server Component puro basta. Se é só interação pós-load → `useQuery` puro basta. O combinado é pro melhor dos dois mundos.

9 O wrapper na prática

Até aqui escrevemos `fetch(...)` direto nos exemplos pra simplificar o ensino. Mas na vida real você quer **uma função só** que centraliza headers, timeout, tratamento de erro tipado, integração com o cache do Next, e — principalmente — injeção automática de token quando estamos no servidor. Sem precisar de duas APIs (uma pro server, outra pro client), sem axios, sem dependência nova.

9.1 — O wrapper completo (um arquivo só)

```

// lib/api-client.ts

// — Erro tipado —
export class ApiError extends Error {
  constructor(
    public readonly status: number,
    public readonly message: string,
  ) {
    super(`API error ${status}: ${message}`);
  }
}

// — Tipos das opções —
type Method = "GET" | "POST" | "PUT" | "PATCH" | "DELETE";

type FetchOptions = {
  method?: Method;
  body?: unknown;
  headers?: Record<string, string>;
  // opções do cache do Next – ignoradas no browser, funcionam no servidor
  next?: { revalidate?: number; tags?: string[] };
  timeoutInMs?: number;
};

```

```
// — A função —
export async function apiFetch<T>({
  path: string,
  options: FetchOptions = {},
}): Promise<T> {
  const { method = "GET", body, headers = {}, next, timeoutInMs = 10_000 } = options;

  // Token: só injeta no SERVIDOR — no browser o cookie vai automaticamente
  let authHeader: Record<string, string> = {};
  if (typeof window === "undefined") {
    const { cookies } = await import("next/headers");
    const token = (await cookies()).get("session_token")?.value;
    if (token) authHeader = { Authorization: `Bearer ${token}` };
  }

  // Detecção por ambiente: no servidor sempre prefixa, no browser sempre relativo
  // Funciona porque o Route Handler espelha o path da API externa
  const url = typeof window === "undefined"
    ? `${process.env.PEDALS_API_URL}${path}`
    : path;

  const response = await fetch(url, {
    method,
    headers: { "Content-Type": "application/json", ...authHeader, ...headers },
    body: body ? JSON.stringify(body) : undefined,
    signal: AbortSignal.timeout(timeoutInMs),
    credentials: "include",
    ...(next ? { next } : {}),
  } as RequestInit);

  if (!response.ok) {
    const text = await response.text().catch(() => "");
    throw new ApiError(response.status, text);
  }

  if (response.status === 204) return undefined as T;

  return response.json() as Promise<T>;
}
```

AS 3 IDEIAS-CHAVE

- **Genérico** `<T>` — quem chama declara o que espera de volta, e `data` sai tipado.
- `typeof window === "undefined"` — detecta servidor pra injetar o token automaticamente. No browser, o cookie vai sozinho via `credentials: "include"`.
- **Detecção por ambiente** — mesma função decide se é rota interna (no browser, mantém relativo) ou externa (no servidor, prefixa a env). Sem heurística no path — só olha onde está rodando.

9.2 — Por que o env não vaza pro browser

`process.env.PEDALS_API_URL` existe **só no servidor**. O Next só envia variáveis pro bundle do browser quando elas começam com `NEXT_PUBLIC_`. Como nossa variável não tem esse prefixo, o browser literalmente não conhece a URL da API externa.

E não precisa: no browser, o `apiFetch` só é chamado com paths que começam com `/api/...`, que são *relativos* e o browser resolve sozinho pro domínio atual. A URL externa só é montada quando o código está rodando no servidor, onde a `env` existe.

MENTAL MODEL: DOIS MUNDOS, MESMA FUNÇÃO, MESMO PATH

```
// no Service (servidor) – chamado por Server Component ou Route Handler
apiFetch<Order[]>("/api/v1/orders")
// → http://api-pedals.com/api/v1/orders (env prefixada, token do cookie injetado)

// no queryFn (browser) – bate no seu próprio Next
apiFetch<Order[]>("/api/v1/orders")
// → /api/v1/orders (relativo, browser resolve pro próprio domínio)
// → cai no Route Handler espelhado em app/api/v1/orders/route.ts
```

Note: o **mesmo path** nos dois lados. O que muda é o ambiente — o `apiFetch` sabe sozinho o que fazer. Pra isso funcionar, o Route Handler precisa **espelhar o path** da API externa.

9.3 — Integração no fluxo completo

Agora os exemplos da seção 8 ficam mais limpos. Olha como o service usa o wrapper:

```
// lib/services/orders.ts
import "server-only"; // 🛡 explode o build se importado num client
import { apiFetch } from "@lib/api-client";

export type Order = {
  id: string;
  status: "pending" | "paid" | "shipped";
  total: number;
};

export type CreateOrderInput = {
  items: { productId: string; quantity: number }[];
};

// Mesmo path que o backend usa – o Route Handler vai espelhar isso
export const OrdersService = {
  listByUser: () =>
    apiFetch<Order[]>("/api/v1/orders", { next: { tags: ["orders"], revalidate: 30 } }),

  getById: (id: string) =>
    apiFetch<Order>(`/api/v1/orders/${id}`, { next: { tags: [`order-${id}`] } }),

  create: (input: CreateOrderInput) =>
    apiFetch<Order>("/api/v1/orders", { method: "POST", body: input }),
};
```

E o Route Handler vira basicamente um forwarder com tratamento de erro — espelhando o path do backend em `app/api/v1/orders/route.ts`:

```
// app/api/v1/orders/route.ts
import { NextResponse } from "next/server";
import { ApiError } from "@lib/api-client";
import { OrdersService } from "@lib/services/orders";
```

```
export async function GET() {
  try {
    return NextResponse.json(await OrdersService.listByUser());
  } catch (err) {
    if (err instanceof ApiError)
      return NextResponse.json({ error: err.message }, { status: err.status });
    return NextResponse.json({ error: "Internal" }, { status: 500 });
  }
}
```

E no client, o TanStack Query usa o **mesmo path** (sem trocar pra `/orders`):

```
// components/orders-list.tsx
"use client";
import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";
import { apiFetch, ApiError } from "@lib/api-client";
// 🛡️ "import type" – apagado no build, NÃO puxa o service pro bundle do browser
import type { Order, CreateOrderInput } from "@lib/services/orders";

export function OrdersList() {
  const qc = useQueryClient();

  const { data, isPending, error } = useQuery<Order[], ApiError>({
    queryKey: ["orders"],
    queryFn: () => apiFetch<Order[]>("/api/v1/orders"), // MESMO path do service
  });

  const createOrder = useMutation({
    mutationFn: (input: CreateOrderInput) =>
      apiFetch<Order>("/api/v1/orders", { method: "POST", body: input }),
    onSuccess: () => qc.invalidateQueries({ queryKey: ["orders"] }),
  });

  if (isPending) return <p>Carregando...</p>;
  if (error?.status === 401) return <p>Faça login.</p>;
  if (error) return <p>Erro: {error.message}</p>;

  return <ul>{data.map(o => <li key={o.id}>#{o.id} – {o.status}</li>)}</ul>;
}
```

9.4 — Route Handler de verdade: params, query, body, validação

Os exemplos até aqui mostraram `GET` simples sem parâmetro nenhum. Na vida real, Route Handler precisa lidar com **dynamic routes** (`/orders/[id]`), **query strings** (`?status=paid&page=2`), **body** de POST/PUT, e **validação** de tudo isso antes de tocar no service. Vamos ver os 4 casos.

Caso 1 — Dynamic route: `GET /api/v1/orders/[id]`

A pasta vira `app/api/v1/orders/[id]/route.ts`. O `id` chega como parâmetro tipado no segundo argumento da função:

```
// app/api/v1/orders/[id]/route.ts
import { NextResponse } from "next/server";
import { ApiError } from "@lib/api-client";
import { OrdersService } from "@lib/services/orders";

// Next 15+: params é Promise. Sempre await.
type RouteContext = { params: Promise<{ id: string }> };
```

```
export async function GET(_req: Request, { params }: RouteContext) {
  const { id } = await params;

  // Validação básica antes de chamar o service
  if (!id || !/^[a-z0-9-]+$/.test(id)) {
    return NextResponse.json({ error: "id inválido" }, { status: 400 });
  }

  try {
    return NextResponse.json(await OrdersService.getById(id));
  } catch (err) {
    if (err instanceof ApiError)
      return NextResponse.json({ error: err.message }, { status: err.status });
    return NextResponse.json({ error: "Internal" }, { status: 500 });
  }
}
```

MUDANÇA IMPORTANTE NO NEXT 15

No Next 14 e anteriores, `params` era um objeto síncrono. No Next 15+, `params` virou **Promise** — você precisa `await` antes de usar. Se você tem código antigo, vai precisar ajustar a tipagem ao atualizar.

Caso 2 — Query string: `GET /api/v1/orders?status=paid&page=2`

Query string vem dentro do `request.url`. Use `URL` nativo pra parsear — não invente regex.

```
// app/api/v1/orders/route.ts
import { NextResponse } from "next/server";
import { OrdersService } from "@lib/services/orders";

export async function GET(request: Request) {
  const { searchParams } = new URL(request.url);

  // searchParams.get retorna string | null — sempre trate o null
  const status = searchParams.get("status") ?? "all";
  const page = Number(searchParams.get("page") ?? "1");
  const perPage = Number(searchParams.get("perPage") ?? "20");

  // Validação manual rápida (versão com zod logo abaixo)
  if (isNaN(page) || page < 1) {
    return NextResponse.json({ error: "page inválido" }, { status: 400 });
  }

  return NextResponse.json(await OrdersService.list({ status, page, perPage }));
}
```

Caso 3 — Body de POST com validação

Pra `POST`, `PUT`, `PATCH` você lê o body com `await request.json()`. **Sempre valide** — `request.json()` devolve `any` e qualquer um pode mandar lixo.

```
// app/api/v1/orders/route.ts (mesmo arquivo do caso 2, função POST)
import { z } from "zod";
import { revalidateTag } from "next/cache";

// Schema vive no Route Handler — domínio dele é "o que aceita HTTP"
```

```

const CreateOrderSchema = z.object({
  items: z.array(z.object({
    productId: z.string().min(1),
    quantity: z.number().int().positive(),
  })).min(1),
});

export async function POST(request: Request) {
  let rawBody: unknown;
  try {
    rawBody = await request.json();
  } catch {
    return NextResponse.json({ error: "JSON inválido" }, { status: 400 });
  }

  const parsed = CreateOrderSchema.safeParse(rawBody);
  if (!parsed.success) {
    return NextResponse.json(
      { error: "validação falhou", issues: parsed.error.issues },
      { status: 400 },
    );
  }

  try {
    // parsed.data está tipado conforme o schema – passa direto pro service
    const order = await OrdersService.create(parsed.data);
    revalidateTag("orders"); // invalida cache da listagem
    return NextResponse.json(order, { status: 201 });
  } catch (err) {
    if (err instanceof ApiError)
      return NextResponse.json({ error: err.message }, { status: err.status });
    return NextResponse.json({ error: "Internal" }, { status: 500 });
  }
}

```

Caso 4 — Validando query string com zod (versão limpa)

Pra query strings complexas (filtros, paginação, ordenação), validação manual fica feia rápido. Faça o mesmo padrão do POST:

```

const ListOrdersQuerySchema = z.object({
  status: z.enum(["pending", "paid", "shipped", "all"]).default("all"),
  page: z.coerce.number().int().positive().default(1),
  perPage: z.coerce.number().int().min(1).max(100).default(20),
});

export async function GET(request: Request) {
  const { searchParams } = new URL(request.url);

  // Object.fromEntries converte URLSearchParams pra objeto simples
  const parsed = ListOrdersQuerySchema.safeParse(
    Object.fromEntries(searchParams)
  );

  if (!parsed.success) {
    return NextResponse.json(
      { error: "query inválida", issues: parsed.error.issues },
      { status: 400 },
    );
  }
}

```

```
// parsed.data já tem tipos certos: status: enum, page: number, perPage: number
return NextResponse.json(await OrdersService.list(parsed.data));
}
```

A DIVISÃO DE RESPONSABILIDADE

- **Route Handler valida o que vem do HTTP** — params, query, body. Garante que o service recebe dado limpo e tipado.
- **Service valida regras de negócio** — "esse usuário pode criar pedido?", "esse produto tem estoque?". Coisas que não dão pra checar só com schema.
- **API externa valida tudo de novo** — porque você nunca confia no que vem antes. Mas isso é problema dela.

Schemas zod vivem perto do Route Handler que os usa — geralmente no *mesmo arquivo* ou em um `schemas.ts` ao lado. Não jogue tudo num arquivo central só "pra organizar" — quando você mudar um endpoint vai ficar pulando entre arquivos sem motivo.

Casos especiais que valem mencionar

Caso	Como tratar
Upload de arquivo	Use <code>await request.formData()</code> em vez de <code>.json()</code> . Pega arquivos com <code>formData.get("file")</code> as <code>File</code> .
Catch-all [...slug]	Params chega como array: <code>params: Promise<{ slug: string[] }></code> . Para <code>/a/b/c</code> , <code>slug = ["a", "b", "c"]</code> .
Status codes específicos	<code>NextResponse.json(data, { status: 201 })</code> pra POST de sucesso. <code>{ status: 204 }</code> com <code>new NextResponse(null, ...)</code> pra DELETE sem corpo.
Headers customizados	<code>NextResponse.json(data, { headers: { "Cache-Control": "no-store" } })</code> .
Cookies (ler ou setar)	<code>cookies()</code> do <code>next/headers</code> dá acesso completo. Setar: <code>(await cookies()).set("nome", "valor", { httpOnly: true })</code> .

9.5 — Tipando retornos do mundo real: o padrão envelope

Nos exemplos até aqui o service devolveu `Order[]` direto — array cru. Funciona, mas **raramente é o que APIs profissionais retornam**. O padrão da indústria (REST com JSON:API, Django REST Framework, NestJS, etc.) é envelopar os dados:

```
// o que sua API REALMENTE devolve
{
  "data": [
    { "id": "1", "title": "...", },
    { "id": "2", "title": "...", }
  ],
  "meta": { "total": 42, "page": 1 },
}
```



```
"links": { "next": "..."}
}
```

O service precisa refletir isso na tipagem. Naive seria tipar como `Order[]` e errar — você vai dar `.map` num objeto e quebrar.

1. Tipa o envelope no service

```
// lib/services/orders.ts
import "server-only";
import { apiFetch } from "@lib/api-client";

export type Order = {
  id: string;
  status: "pending" | "paid" | "shipped";
  total: number;
};

// Envelope reutilizável – todo endpoint de listagem usa esse formato
export interface PaginatedResponse<T> {
  data: T[];
  meta: { total: number; page: number; perPage: number };
}

export const OrdersService = {
  listByUser: () =>
    apiFetch<PaginatedResponse<Order>>("/api/v1/orders", {
      next: { tags: ["orders"], revalidate: 30 },
    }),
};
```

2. Desembrulhe no boundary com o `select` do TanStack

Agora o problema: o `useQuery` vai te dar `PaginatedResponse<Order>` e você vai escrever `data?.data.map(...)` em todo componente. Feio e propenso a erro.

A solução elegante é o `select` — ele **transforma o dado depois do cache**, rodando uma vez por resultado (memoizado). O cache continua armazenando o envelope completo (caso outro componente queira o `meta` pra paginação), mas o componente recebe só o array já desembrulhado.

```
// components/orders-list.tsx
"use client";
import { useQuery } from "@tanstack/react-query";
import { apiFetch, ApiError } from "@lib/api-client";
import type { Order, PaginatedResponse } from "@lib/services/orders";

export function OrdersList() {
  const { data: orders, isPending, error } = useQuery<
    PaginatedResponse<Order>, // TQueryFnData – o que a função retorna (cache)
    ApiError,                 // TError
    Order[]                    // TData – o que o componente recebe (após select)
  >({
    queryKey: ["orders"],
    queryFn: () => apiFetch<PaginatedResponse<Order>>("/api/v1/orders"),
    select: (response) => response.data, // desembrulha aqui, uma vez só
  });

  if (isPending) return <p>Carregando...</p>;
```

```
if (error) return <p>Erro: {error.message}</p>;

// orders aqui é Order[], não PaginatedResponse — direto, limpo
return <ul>{orders.map(o => <li key={o.id}>#{o.id}</li>)}</ul>;
}
```

OS 3 GENÉRICOS DO USEQUERY

- `TQueryFnData` — o que o `queryFn` retorna. Vai pro cache.
- `TError` — o tipo do erro. Use sua `ApiError` pra ganhar autocomplete em `err.status`.
- `TData` — o tipo final que o componente recebe. Se você usa `select`, é o retorno dele. Se não, é igual a `TQueryFnData`.

Quando você passa os três, o TypeScript valida que o `select` está realmente transformando de `TQueryFnData` pra `TData`. Erra a transformação, o build quebra. Esse é o nível de segurança que vale a complexidade.

3. Outros casos do `select` que merecem destaque

O `select` não serve só pra desembulhar — ele é o lugar pra **qualquer derivação** de dados pré-renderização:

```
// Filtrar antes do componente ver
select: (res) => res.data.filter(o => o.status === "paid"),

// Calcular agregado (memoizado — só recalcula se response mudar)
select: (res) => ({
  orders: res.data,
  totalValue: res.data.reduce((sum, o) => sum + o.total, 0),
}),

// Pegar só um campo — componentes que só usam isso não re-renderizam
// se outros campos do envelope mudarem
select: (res) => res.meta.total,
```

POR QUE ISSO É MELHOR DO QUE TRANSFORMAR NO QUERYFN

Você poderia transformar dentro do `queryFn` antes de retornar. Mas aí o cache guarda só o transformado — perdeu acesso ao envelope original. Componentes diferentes podem precisar de visões diferentes do mesmo dado (um precisa do array, outro do `meta.total` pra paginação). O `select` mantém o cache canônico e dá views customizadas por componente. **Cache compartilhado, projeções específicas** — esse é o ponto.

9.6 — Onde tratar erros (cada camada tem o seu lugar)

O `apiFetch` lança `ApiError`. Quem pega depende da camada:

Camada	Como tratar
Server Component	Deixa lançar. O Next captura via <code>app/.../error.tsx</code> automaticamente (que precisa ser <code>"use client"</code> porque tem <code>reset()</code>). Use try/catch só pra casos específicos como <code>redirect("/login")</code> no 401.

Camada	Como tratar
Route Handler	Try/catch obrigatório com <code>instanceof ApiError</code> . Repassa <code>err.status</code> pro cliente em vez de devolver HTML genérico de 500.
<code>useQuery</code>	Nunca try/catch. Use <code>const { error } = useQuery<T, ApiError>(...)</code> e cheque <code>error?.status</code> .
<code>useMutation</code>	Callback <code>onError: (err) => { ... }</code> . <code>err</code> vem tipado como <code>ApiError</code> se você passou o genérico.

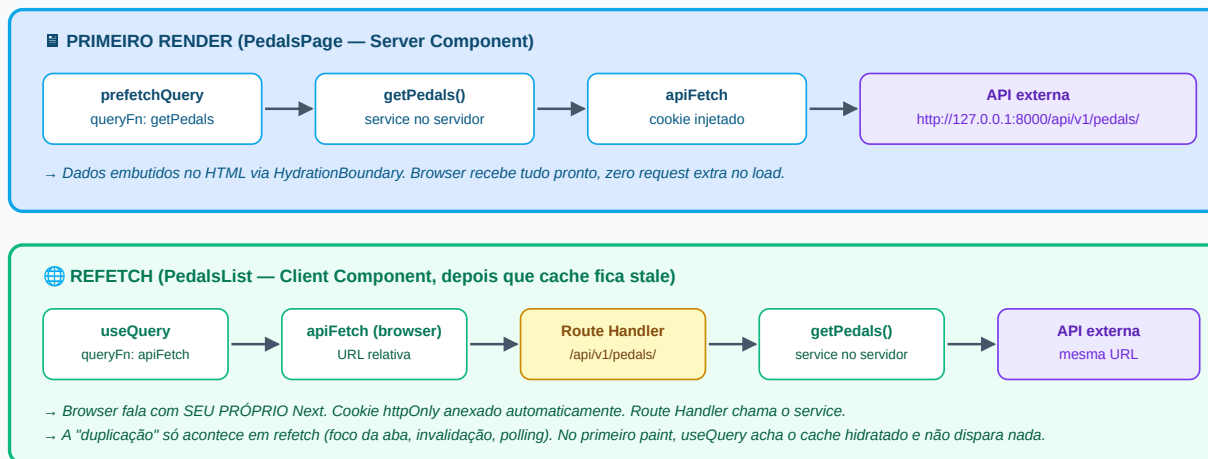
O QUE ESSE WRAPPER DELIBERADAMENTE NÃO FAZ

Sem retry automático, sem refresh de token, sem interceptor global. Tudo isso é resolvível em cima do `apiFetch` quando o problema aparecer (envolver com retry, tratar 401 antes do throw, logar no início da função). **Começa simples, adiciona complexidade quando precisar** — é o oposto de partir do axios com 12 plugins "por garantia".

9.7 — "Mesma chamada, ambientes diferentes" — o insight que destranca tudo

Quando você olha pro Server Component prefetchando e pro Client Component fazendo `useQuery`, parece que estão fazendo a mesma coisa duas vezes. **Não estão.** É a mesma *função*, em *ambientes diferentes*, com fluxos completamente diferentes.

A página `PedalsPage` carrega uma vez. Veja o que cada caminho faz:



A PERGUNTA CERTA NÃO É "POR QUE DUAS CHAMADAS?"

É **"em qual ambiente cada uma roda?"**. O Server Component chama o service direto porque *já está no servidor* — atalho perfeito. O Client Component não pode chamar o service direto (sem cookie, sem env, com CORS) — então fala com o BFF, que está no servidor, que aí sim chama o service. Mesma função, dois caminhos, segurança preservada.

9.8 — Checklist de erros comuns

Os bugs que sempre aparecem quando a gente começa a usar esse padrão. Confira **todos** antes de gritar "tá quebrado".

Sintoma / código suspeito	Diagnóstico
<pre>queryFn: () => apiFetch("api/v1/pedals/")</pre>	Falta a barra inicial. Sem <code>/</code> , browser resolve relativo à URL atual da página. Se usuário está em <code>/pedals</code> , vira <code>/pedals/api/v1/pedals/</code> → 404.
Request quebra com <pre>undefined/api/v1/...</pre>	Service rodando no browser por acidente. <code>process.env.PEDALS_API_URL</code> é <code>undefined</code> no client. Adicione <code>import "server-only"</code> no service pra detectar isso em build time.
Client retorna 404 ao chamar <pre>/api/v1/pedals/</pre>	Falta o Route Handler. Crie <code>app/api/v1/pedals/route.ts</code> que importa o service e devolve o JSON. Sem ele, o Next não conhece a rota.
<pre>import { getPedals } from "@/services/..." em "use client"</pre>	Vazamento de service pro bundle. Use <code>import type { ... }</code> se só precisa do tipo. Com <code>"server-only"</code> , o build quebra com mensagem clara.
Erro <code>Module not found: next/headers</code> em build	Algum client component importou direta ou indiretamente algo que usa <code>cookies()</code>. O <code>apiFetch</code> evita isso com <code>await import("next/headers")</code> dinâmico — confirme que está dinâmico, não estático no topo.
<pre>useQuery({ queryKey: ["pedals"], queryFn: ... })</pre> sem genérico	Funciona, mas <code>data</code> é <code>unknown</code>. Use <code>useQuery<Pedal[]></code> , <code>ApiError</code> ou tipe o retorno do <code>apiFetch<Pedal[]></code> .
Caminhos do service e do client diferentes (<code>/orders</code> vs <code>/api/v1/orders</code>)	Funciona com a heurística antiga (<code>startsWith("/api/")</code>) mas é frágil. Padrão recomendado: mesmo path dos dois lados + Route Handler espelhando estrutura do backend.

10 Receitas prontas

"Blog post público" (1 leitura, sem interação)

→ **Server Component puro.** Sem Route Handler, sem TanStack. Importe o service, `await`, renderiza.

"Lista de produtos com filtros e paginação"

→ **Server Component faz o primeiro paint + HydrationBoundary.** Client component com TanStack Query cuida dos filtros (`queryKey` muda quando filtro muda). Route Handler em `/api/products` serve os refetches.

"Dashboard logado com cookie httpOnly"

→ **Route Handler obrigatório.** O cookie só é legível no servidor. TanStack Query no client bate em `/api/me`, `/api/orders`, etc. O Route Handler lê o cookie, troca por token da API externa, devolve o JSON.

"Criar / editar / deletar"

→ **Server Action** se for via form do próprio site. **Route Handler (POST/PUT/DELETE)** se for chamado via TanStack `useMutation` de um componente complexo, ou se outro cliente (mobile) precisar do mesmo endpoint.

"Webhook do Stripe"

→ **Route Handler.** Sem opção. `POST /api/webhooks/stripe`, valide a assinatura, processe.

"API pública de terceiro, sem segredo, com CORS ok"

→ **Pode bater direto do client.** Não invente BFF só por inventar. Ex.: API pública do GitHub para mostrar contribuições, OpenWeather com chave pública, etc. Se for chave *secreta*, aí sim BFF.

11 Anti-patterns que vão te queimar

1. SERVER COMPONENT CHAMANDO O PRÓPRIO /API DO NEXT

```
// ❌ ERRADO - loopback inútil
export default async function Page() {
  const res = await fetch("http://localhost:3000/api/posts");
  const posts = await res.json();
  return <List posts={posts} />;
}
```

Servidor falando HTTP com servidor (ele mesmo) por nada. Importe o service direto.

2. CHAVE SECRETA NO CLIENT

```
// ❌ NEXT_PUBLIC_ vai pro browser. Não bota secret aqui.
fetch("https://api.terceiro.com", {
  headers: { Authorization: `Bearer ${process.env.NEXT_PUBLIC_API_KEY}` }
});
```

Se a variável começa com `NEXT_PUBLIC_`, ela é pública. Pra secret, use Route Handler que lê `process.env.API_KEY` (sem prefixo).

3. USEQUERY DENTRO DE UM SERVER COMPONENT

Não compila. `useQuery` é hook, exige client. Server Component usa o service direto.

4. QUERYCLIENT COMPARTILHADO ENTRE REQUESTS NO SERVIDOR

Criar `const client = new QueryClient()` no top-level do módulo é vazamento de cache entre usuários. **Sempre** instancie por request (no Server Component, dentro da função; no client, com `useState`).

5. ROUTE HANDLER VIROU COPY-PASTE DO FRONT

Se seu `route.ts` é só "recebe, repassa, devolve" sem nenhuma transformação, auth ou secret no meio, ele provavelmente não devia existir. Avalie chamar a API direto ou levar a chamada pro Server Component.

6. ESQUECER DE VALIDAR INPUT EM SERVER ACTION

Server Action é endpoint público disfarçado. Use `zod` ou similar no começo da função, igual você faria num Route Handler. `"use server"` não autentica nada.

12 Cheatsheet final

🎯 Como decidir, em 5 perguntas

1. **Tem secret na chamada?** → Sim: BFF (Route Handler). Não: pode ir direto.
2. **O dado aparece no primeiro paint?** → Sim: Server Component. Não: client + TanStack.
3. **É uma mutação chamada do próprio front?** → Sim: Server Action é o caminho mais simples.
4. **Algo de fora (mobile, webhook, outro serviço) chama?** → Route Handler obrigatório.
5. **Precisa de SEO + interação rica depois?** → Server Component prefetcha → HydrationBoundary → client com `useQuery`.

Cenário	Server Comp	Route Handler (BFF)	TanStack Query
Página estática de leitura	✓ usa	não precisa	não precisa
Lista filtrável com paginação	prefetch	✓ serve refetch	✓ no client
API com secret/token	se for paint inicial	✓ obrigatório	✓ se for client
Form de criação	—	opcional (mobile)	Server Action resolve
Webhook	—	✓ obrigatório	—
API pública + CORS ok	se for paint inicial	não precisa	✓ direto da API
Polling / realtime feel	—	✓ serve	✓ refetchInterval
Auth com cookie httpOnly	leitura inicial	✓ obrigatório	✓ pra dados pós-login

13 Fontes

Tudo nesse documento vem das fontes abaixo, consultadas em maio/2026. Sempre que possível, oficial primeiro.

Next.js — Building APIs with Next.js (oficial)

Explica Route Handlers como BFF, proxy, agregação, webhooks.
<https://nextjs.org/blog/building-apis-with-nextjs>

Next.js — App Router Glossary (oficial)

Definições oficiais de Server Component, Server Action, Route Handler, RSC Payload.
<https://nextjs.org/docs/app/glossary>

TanStack Query — Advanced Server Rendering (oficial)

Padrão de prefetch + dehydrate + HydrationBoundary para App Router.
<https://tanstack.com/query/latest/docs/framework/react/guides/advanced-ssr>

TanStack Query — Server Rendering & Hydration (oficial)

Conceitos base de SSR, initialData vs hydration, QueryClient por request.
<https://tanstack.com/query/latest/docs/framework/react/guides/ssr>

Server Actions vs Route Handlers — MakerKit

A regra prática mais limpa que vi de quando usar qual.
<https://makerkit.dev/blog/tutorials/server-actions-vs-route-handlers>

Next.js Route Handlers: The Complete Guide — MakerKit

Tirou de letra os anti-patterns (loopback HTTP, secret no client).
<https://makerkit.dev/blog/tutorials/nextjs-api-best-practices>

BFF Pattern with Next.js API Routes — Zafer Bozkurt, Medium

Implementação concreta do padrão BFF em Next, inclusive token forwarding via cookie.
<https://medium.com/digigeek/bff-backend-for-frontend-pattern-with-next-js-api-routes-secure-and-scalable-architecture-d6e088a39855>

Why Next.js is the Ultimate Tool for BFF — Carlos Orioli, DEV

Padrão Service → Route Handler → Component que apresentamos na seção 6.
<https://dev.to/carlosorioli/why-nextjs-is-the-ultimate-tool-for-the-bff-backend-for-frontend-pattern-41ej>

Building Fully Hydrated SSR App with Next + TanStack — Sangwin Gawande, Medium

Exemplo completo de prefetch + hydrate funcionando ponta a ponta.
<https://sangwin.medium.com/building-a-fully-hydrated-ssr-app-with-next-js-app-router-and-tanstack-query-5970aaf822d2>

TanStack Query Discussion #6267 — App Router + Hydration Boundary

Discussão oficial no GitHub com exemplos de quando o padrão prefetch+hydrate compensa.
<https://github.com/TanStack/query/discussions/6267>

Setting Up Proxy API Routes in Next.js — Next SaaS Pilot

Quando o BFF é só pra escapar de CORS / esconder API key, com gotchas de POST stream.
<https://www.nextsaaspilot.com/blogs/nextjs-proxy-api-route>

Fim da aula. Você não é burro — Next.js é genuinamente complicado nessa parte. Esse documento existe pra você não ter que reler 11 blogs toda vez.